

SMtrack v2 Backend — Tech Stack & System Architecture

This document summarizes the **actual current state of the code** (based on `package.json`, `prisma/schema.prisma` + migrations, `src/app.module.ts`, `src/main.ts`, `docker-compose.yml`) rather than paraphrasing `CLAUDE.md`. Where the real code diverges from the original `CLAUDE.md` description, it is called out explicitly in each section.

1. System Overview

A backend for ingesting, processing, and storing IoT telemetry/log data (medical refrigerators/freezers with temperature/humidity/door probes) and raising alerts when readings go out of range. MQTT is the primary ingestion channel; a REST API + SSE stream serve the dashboard, and FCM delivers push notifications to mobile.

2. Tech Stack

Category	Actual technology in use
Framework	NestJS v11 (<code>@nestjs/core</code> , <code>common</code> , <code>platform-express</code> , <code>microservices</code> , <code>config</code> , <code>event-emitter</code> , <code>schedule</code> , <code>swagger</code> , <code>terminus</code> , <code>passport</code>) — hybrid app: HTTP + MQTT microservice + RabbitMQ microservice x2
ORM / DB	Prisma v7.8 (new <code>prisma-client</code> generator, output to <code>src/generated/prisma</code>) + <code>@prisma/adapters-pg</code> (driver adapter) + raw <code>pg</code> v8 for specific tasks (e.g. <code>pg-copy-streams</code> for bulk export) on PostgreSQL + TimescaleDB extension
MQTT	<code>mqtt</code> v5.10
Message queue	RabbitMQ via <code>amqplib</code> + <code>amqp-connection-manager</code> (used through <code>@nestjs/microservices</code> RMQ transport) — split into 2 microservices on separate queues (general events vs. high-volume log/telemetry)
Cache	Redis via <code>ioredis</code>
Push notification	<code>firebase-admin</code> v13 (FCM)
Auth	<code>passport</code> + <code>passport-jwt</code> — two separate schemes : <code>jwt</code> (user login) and <code>device</code> (device token)
Object storage	<code>@aws-sdk/client-s3</code> + <code>@aws-sdk/lib-storage</code> — S3-compatible (MinIO in local dev), 3 independent configs (archive, device image, firmware)

Category	Actual technology in use
Observability	Full OpenTelemetry SDK (<code>sdk-node</code> , auto-instrumentations, OTLP HTTP exporter for both trace and metrics) + <code>nestjs-pino</code> / <code>pino</code> / <code>pino-http</code> for structured logging
Validation	<code>class-validator</code> + <code>class-transformer</code>
Testing	Jest v29 + <code>ts-jest</code> + <code>supertest</code> + <code>@nestjs/testing</code>
Other	<code>mongodb</code> v7.5 present in devDependencies (no actual usage found elsewhere in the code)

Diverges from CLAUDE.md : Prisma is now on v7 (generator provider changed to `prisma-client` , not the original `prisma-client-js`); object storage (S3/MinIO) and a full OpenTelemetry observability stack exist and are not mentioned in `CLAUDE.md` at all.

3. Module Structure (`src/`)

From `app.module.ts` , grouped into 3 categories:

Infrastructure `Prisma` , `Redis` , `Rabbitmq` , `Auth` , `Cache` (common), `Observability`

Core vertical slice `Sse` , `Telemetry` , `Mqtt` , `Fcm` , `Notification` , `Device` , `Audit` , `Probe` , `DeviceConfig` , `DeviceRepair` , `Warranties` , `Firmware` , `UserAudit`

Supporting `Logday` , `Graph` , `Backup` , `Health` , `Consumer` (event consumer), `LogConsumer` (log/telemetry consumer on a separate queue), `Mobile`

Diverges from CLAUDE.md : modules not mentioned in `CLAUDE.md` but present in the code — `audit` , `device-config` , `device-repair` , `warranties` , `firmware` , `user-audit` , `observability` — plus `consumer` / `log-consumer` being two separate modules (not a single combined `consumer/` as originally described). The `mobile` module exists as originally described.

4. Data Model (Prisma schema)

The most significant design divergence from `CLAUDE.md` : the real schema splits **Devices** (a logical install point, e.g. "ICU bed 3 refrigerator") from **Hardware** (the physical box, keyed by `serial`) instead of combining them into one model, and there is **no Users /hospital table** — every actor is derived purely from the JWT payload.

- **Devices** — install point; `staticName` is the permanent identity, `serial` is just a pointer to whichever box is currently installed (denormalized from the active assignment). Must never be written directly — only through `DeviceSwapService` .
- **Hardware** — the physical box, keyed by `serial` ; carries firmware/token/repair history/warranty, which always follow the box.

- **DeviceAssignments** — history of which box was installed at which install point during which time range (source of truth for hardware swaps; overlapping ranges prevented via a partial unique index in the migration).
- **Probes** , **Configs** — tied to `Devices.id` (site-specific settings; swapping in a new box at the same install point restores these immediately).
- **Repairs** , **Warranties** — tied to `Hardware.serial` (follow the box, not the install point).
- **LogDays** — telemetry, **TimescaleDB hypertable on `sendTime`** , with nullable `deviceId` / `probeId` (fallback for a box not yet installed anywhere).
- **Notifications** — **TimescaleDB hypertable on `createAt`** , with `deliveredSse` / `deliveredFcm` as described in `CLAUDE.md` , **plus** `isRead` / `readAt` (user-side read state, distinct from delivery status) and `category` / `severity` (computed at creation by a classifier: `TEMP|DOOR|INTERNET|PLUG|SDCARD|REPORT|OTHER` , `critical|warning|info`).
- **LogDayArchive** + **ArchiveExport** / **ArchiveRestore** — `LogDays` rows older than the retention window are exported to S3/MinIO and removed from the hypertable; the two audit tables prevent duplicate export/restore.
- **DeviceAudit** / **UserAudit** — audit trail, also hypertables, with no FK to any entity (so audit rows survive even if the entity is deleted).
- **Firmwares** — OTA file metadata; actual files live in a dedicated S3 bucket.

Actual TimescaleDB hypertables/retention policies (from `prisma/migrations/*/migration.sql`):

Table	Hypertable on	Retention
<code>LogDays</code>	<code>sendTime</code>	— (aged out via the archive job instead)
<code>LogDayArchive</code>	<code>sendTime</code> , 1-month chunks	—
<code>notifications</code>	<code>createAt</code>	—
<code>device_audit</code>	<code>createAt</code>	6 months
<code>user_audit</code>	<code>createAt</code>	6 months

No continuous aggregates exist in the current system (no TimescaleDB continuous-aggregate rollups have been set up yet).

5. Bootstrap / Runtime Architecture

From `src/main.ts` — hybrid application:

1. `NestFactory.create(AppModule, { bufferLogs: true })` (guarded by a 30-second timeout to prevent hanging if DB/MQTT connection fails without rejecting).
2. Global setup: `ValidationPipe (transform, whitelist)` , `HttpExceptionFilter` , `ResponseInterceptor` (wraps every response as `{ success, message, data, timestamp,`

`statusCode }`), global prefix `log`, permissive CORS (`origin: '*'`, no cookies — clients send `fetch/EventSource` with an `Authorization` header).

3. Swagger is enabled only outside production, mounted at `/docs`, with 2 bearer schemes (`jwt`, `device`).
4. `connectMicroservice` attaches **3 microservices** to the same app: MQTT, RabbitMQ (general events), RabbitMQ (log/telemetry on a separate queue so high volume doesn't block the other queue).
5. `startAllMicroservices()` (same timeout guard), then `listen()`.

6. Ingestion Paths

Log/telemetry data can enter through 3 paths, as described in `CLAUDE.md` and confirmed in `src/consumer/` (which has a `log-consumer` separate from `notification-consumer`):

- **MQTT** (primary) — subscribes to device topics.
- **HTTP** — `POST /logday` as a fallback.
- **RabbitMQ consume** — a separate queue for other upstream gateways/services.

All 3 paths funnel into a single entry point (`TelemetryService.ingest()`) — no duplicated insert logic.

7. Notification Fan-out

When a new notification is created, it fans out through 2 independent channels (a failure in one never blocks the other), as designed in `CLAUDE.md`:

- **SSE** — broadcast to any client/dashboard with an open stream.
- **FCM** — a single broadcast to topic `device_{serial}` (never looped per-token).

The outcome of each channel is recorded back onto `deliveredSse` / `deliveredFcm` on the `Notifications` row itself, alongside `category` / `severity` computed by the classifier at creation time — a detail not covered in `CLAUDE.md`.

8. Infrastructure (`docker-compose.yml`)

Local full stack (`docker compose up -d --build`):

Service	Image
<code>timescaledb</code>	<code>timescale/timescaledb:latest-pg17</code>
<code>redis</code>	<code>redis:7-alpine</code>
<code>rabbitmq</code>	<code>rabbitmq:3-management-alpine</code>
<code>mosquitto</code>	<code>eclipse-mosquitto:2</code>

Service	Image
minio	minio/minio:RELEASE.2025-04-22... (S3-compatible object storage, standing in for real S3 in dev)

The full observability stack (`otel-collector` , `tempo` , `loki` , `prometheus` , `alertmanager` , `alloy` , `grafana`) is pulled in via `include:` from a separate `observability/docker-compose.yml` file, not duplicated here. Grafana runs on port 3001 (to avoid colliding with the app on 3000).

`migrate` and `minio-init` are one-shot jobs that run before `app` starts (schema migration + required bucket creation).

Entirely absent from the original CLAUDE.md : MinIO/S3 object storage and the whole observability stack — both were added after the original description was written.

9. Auth

Two JWT schemes, separated by audience:

- `jwt` — user login (dashboard).
- `device` — device token (for endpoints called directly by devices, e.g. HTTP fallback ingestion).

There is no `Users` model in the database — actor fields (`actorId` / `actorName` / `actorRole`) recorded in the audit tables come entirely from the JWT payload.